CSCE 3530 FINAL PROJECT

Design and Validation of a Small Virtual LAN in GNS3

Integrated DHCP, DNS, NAT, HTTP, ICMP, ARP, TCP, and UDP Services

Student	Hamza Shakur
Instructor	Ali Zarafshani
Course	CSCE 3530 – Introduction to Computer Networks
Semester	Summer 2026
Platform	GNS3, Ubuntu Linux, VPCS, Apache2, dnsmasq

Final Project Report

Prepared for academic submission

Executive Summary

This project implements and validates a small routed local area network in GNS3. An Ubuntu virtual machine operates as the network gateway and service host for the 192.168.10.0/24 subnet. The design combines address assignment, name resolution, web hosting, packet forwarding, and protocol-level testing in one compact laboratory environment. Two VPCS endpoints and one server node connect through a virtual Ethernet switch, while the Ubuntu router provides the default gateway at 192.168.10.1.

The implementation demonstrates the interaction of the major protocols introduced in CSCE 3530. DHCP supplies host configuration, DNS translates domain names, NAT allows private addresses to reach external networks, HTTP serves a webpage, ICMP verifies reachability, ARP resolves local hardware addresses, TCP supports reliable web sessions, and UDP supports lightweight DHCP and DNS exchanges. Evidence in the screenshots confirms that each required service was configured and tested successfully.

Project Objective	Implementation Evidence	Result
Build a functional virtual LAN	GNS3 topology with Ubuntu router, switch, PC1, PC2, and server	Completed
Provide automatic addressing	dnsmasq DHCP pool 192.168.10.100–150	Completed
Provide DNS forwarding	dnsmasq forwards to 8.8.8.8 and 1.1.1.1	Completed
Provide Internet access	iptables forwarding and masquerading	Completed
Host a web service	Apache2 active and default webpage loaded	Completed
Validate protocol behavior	Ping, nslookup, browser test, and tcpdump	Completed

Overall result: the virtual LAN met the functional requirements and showed end-to-end connectivity from the internal hosts to both local and Internet resources.

Table of Contents

Executive Summary 2
Table of Contents 3
Report Organization 4
1. Introduction and Project Objectives 5
Primary Objectives 5
Success Criteria 5
2. Network Topology and Architecture 6
3. IP Addressing Plan 7
Addressing Rationale 7
Addressing Verification 7
4. Ubuntu Router and NAT Configuration 8
Required Router Functions 8
5. DHCP and DNS Service Configuration 9
6. Protocol Analysis: DHCP 10
DORA Process 10
Project Behavior 10
Operational Importance 10
7. Protocol Analysis: DNS 11
8. Protocol Analysis: HTTP and Apache 12
9. HTTP Browser Validation 13
Observed Result 13
10. Protocol Analysis: ICMP and Connectivity Testing 14
11. Protocol Analysis: ARP 15
ARP in This Project 15
12. Protocol Analysis: TCP and UDP 16
TCP Operation 16
UDP Operation 16
13. DHCP Lease Verification on PC2 17
14. Router Interface Verification 18
Verification Checklist 18
15. Packet Capture and Results Analysis 19
16. Integrated Testing and Results 20
17. Challenges, Lessons Learned, and Conclusion 21
Key Challenges 21
Lessons Learned 21
Conclusion 21
References 21

The table above is an automatic Microsoft Word field. Open the report in Word, right-click the table, and select “Update Field” → “Update entire table” if the page numbers do not refresh on first opening.

Report Organization

- Network requirements and design rationale
- Topology and IP addressing plan
- Ubuntu router and service configuration
- Protocol explanations for DHCP, DNS, NAT, HTTP, ICMP, ARP, TCP, and UDP
- Testing evidence and packet-capture analysis
- Challenges, lessons learned, conclusion, and references

1. Introduction and Project Objectives

Modern networks depend on multiple protocols working together rather than on a single service. This project was designed to demonstrate that dependency in a controlled virtual environment. GNS3 was used to assemble the network, Ubuntu Linux was used as the router and service platform, VPCS nodes represented client computers, and an additional node represented an internal server.

The project emphasizes both configuration and verification. A configuration is not considered complete merely because a command was entered; it must be validated through observable results. For that reason, the report pairs each implementation area with a screenshot showing the actual output of the system.

Primary Objectives

1. Create the 192.168.10.0/24 internal LAN and connect all devices through a virtual Ethernet switch.
2. Configure the Ubuntu VM as the default gateway at 192.168.10.1/24.
3. Use dnsmasq to provide DHCP leases and DNS forwarding.
4. Enable IP forwarding and NAT masquerading so internal hosts can reach external networks.
5. Install and start Apache2, then confirm that the web page is accessible from a browser.
6. Use ping, nslookup, show ip, hostname -I, and tcpdump to verify addressing, reachability, name resolution, and packet flow.

Success Criteria

- All internal devices can communicate with the gateway.
- At least one host receives a DHCP lease in the configured pool.
- DNS lookups return valid records.
- The Apache service reports an active state and its webpage loads.
- ICMP packets reach both the gateway and an Internet destination.
- Packet capture displays ARP and ICMP traffic traversing the router interface.

2. Network Topology and Architecture

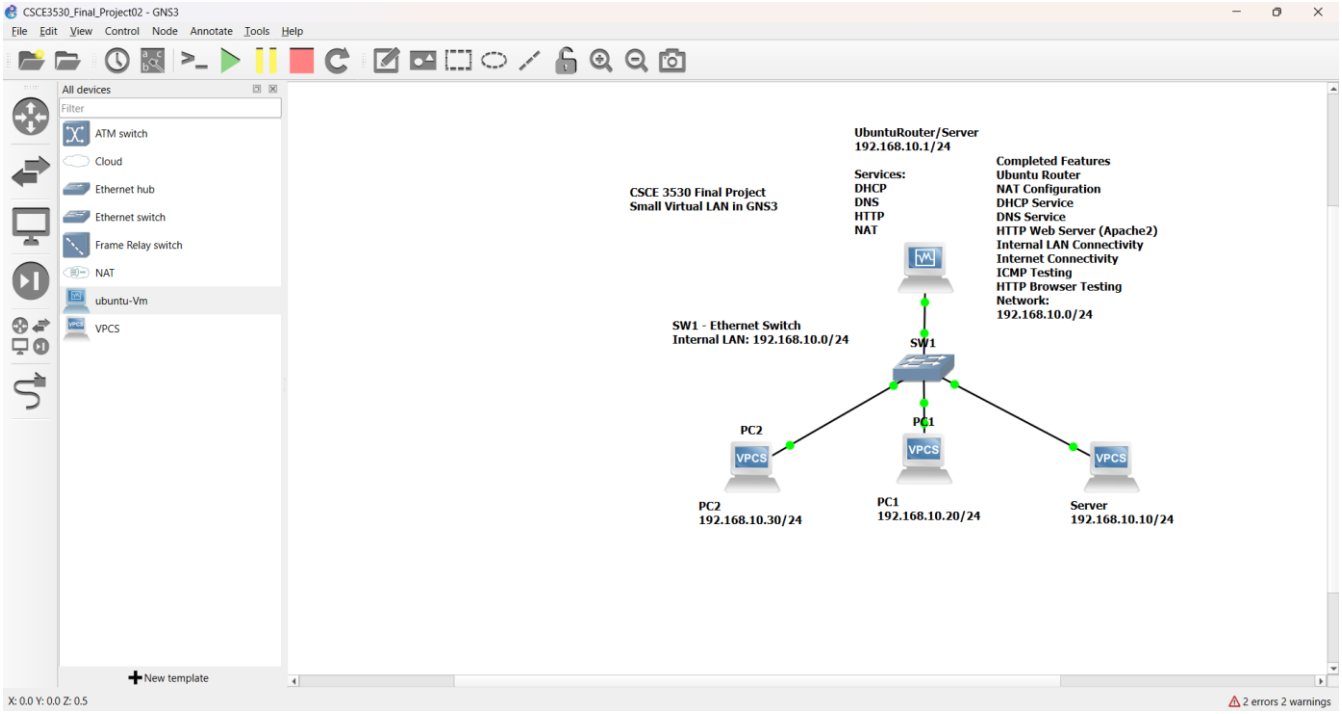

Figure 1. GNS3 topology showing the Ubuntu router/server, Ethernet switch, PC1, PC2, and internal server on 192.168.10.0/24.

The topology follows a star design. SW1 is the central Layer 2 connection point, while the Ubuntu VM performs Layer 3 routing and hosts the infrastructure services. PC1 and PC2 represent user endpoints. The server node is assigned 192.168.10.10/24, PC1 is assigned 192.168.10.20/24, and PC2 is shown as 192.168.10.30/24 in the design diagram. During DHCP testing, PC2 also demonstrates dynamic configuration from the DHCP pool.

This design is appropriate for a small laboratory because it separates the switching role from the routing/service role while remaining easy to troubleshoot. All local devices share the same broadcast domain, so ARP can be observed directly, and the Ubuntu gateway provides a single controlled point for forwarding, NAT, DNS, DHCP, and HTTP services.

3. IP Addressing Plan

Device / Function	Address	Mask / Prefix	Default Gateway	Assignment
Internal network	192.168.10.0	255.255.255.0 (/24)	N/A	Network ID
Ubuntu router / DNS / DHCP / HTTP	192.168.10.1	255.255.255.0 (/24)	Upstream interface	Static
Internal server node	192.168.10.10	255.255.255.0 (/24)	192.168.10.1	Static
PC1	192.168.10.20	255.255.255.0 (/24)	192.168.10.1	Static
PC2 design address	192.168.10.30	255.255.255.0 (/24)	192.168.10.1	Static in topology
DHCP client example	192.168.10.100	255.255.255.0 (/24)	192.168.10.1	Dynamic
DHCP pool	192.168.10.100–150	255.255.255.0 (/24)	192.168.10.1	12-hour leases

Addressing Rationale

The /24 prefix provides 254 usable host addresses, which is more than sufficient for the project while keeping subnet calculations simple. Infrastructure addresses are placed at the lower end of the subnet, and the dynamic range begins at .100. This separation reduces the possibility of conflicts between manually assigned systems and DHCP clients.

The gateway and DNS address are both 192.168.10.1 because the Ubuntu VM performs both roles. Clients therefore need only one infrastructure address for routing and local DNS forwarding. The DHCP lease duration is 12 hours, which is reasonable for a temporary laboratory and allows addresses to be reclaimed without excessive renewal traffic.

Addressing Verification

- PC1 reports 192.168.10.20/24, gateway 192.168.10.1, and DNS 192.168.10.1.
- PC2 obtains 192.168.10.100/24 from the configured pool during DHCP testing.
- The Ubuntu router reports 192.168.10.1 on its internal interface.
- All addresses belong to the same 192.168.10.0/24 LAN and can communicate directly through SW1.

4. Ubuntu Router and NAT Configuration

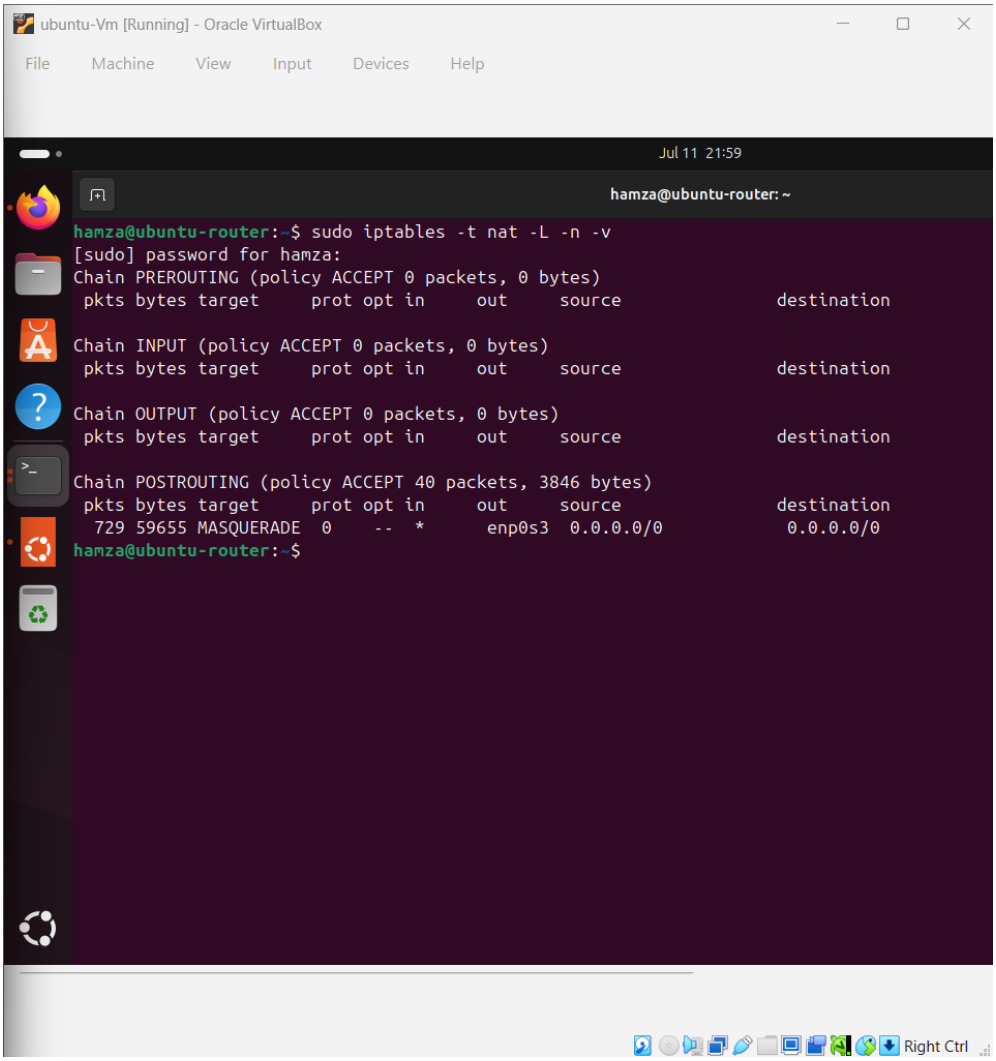

Figure 2. iptables rules enabling forwarding and NAT masquerading on the Ubuntu router.

Network Address Translation allows private RFC 1918 addresses on 192.168.10.0/24 to communicate with external networks. The screenshot shows forwarding rules that permit established return traffic and traffic entering from the internal interface. It also shows a POSTROUTING MASQUERADE rule on the external interface. Masquerading replaces the private source address with the router's external address and tracks the connection so response packets can be returned to the correct internal host.

Required Router Functions

- IPv4 forwarding must be enabled in the Linux kernel.
- The FORWARD chain must permit traffic from the internal interface and permit established or related return traffic.
- The NAT table must apply MASQUERADE to packets leaving the external interface.
- The internal clients must use 192.168.10.1 as their default gateway.

The configuration follows the principle of stateful forwarding: new outbound sessions originate from the internal network, while inbound packets are accepted when they belong to an already established or related session. This is more controlled than allowing unrestricted forwarding in both directions.

5. DHCP and DNS Service Configuration

Figure 3. dnsmasq configuration for interface enp0s8, DNS forwarding, and the DHCP address pool.

dnsmasq was selected because it combines a lightweight DHCP server and DNS forwarder in one service. The configuration binds the service to the internal interface and listens on 192.168.10.1. The bind-interfaces directive limits the service to the intended interface, reducing accidental exposure on other network interfaces.

Directive	Configured Value	Purpose
interface	enp0s8	Serve the internal GNS3 LAN.
listen-address	192.168.10.1	Listen on the router's LAN address.
bind-interfaces	Enabled	Restrict dnsmasq to configured interfaces.
no-resolv	Enabled	Ignore the host resolver file and use explicit upstream servers.
server	8.8.8.8 and 1.1.1.1	Forward external DNS questions.
dhcp-range	192.168.10.100–150, /24, 12h	Define the dynamic address pool and lease time.
dhcp-option 3	192.168.10.1	Provide the default gateway.
dhcp-option 6	192.168.10.1	Provide the DNS server address.
dhcp-authoritative	Enabled	Indicate that this server is authoritative for the subnet.

6. Protocol Analysis: DHCP

Dynamic Host Configuration Protocol automates the delivery of essential host parameters. Without DHCP, each endpoint would require manual configuration of its address, subnet mask, gateway, and DNS server. DHCP normally uses UDP port 67 on the server and UDP port 68 on the client because a client may not yet have an IP address when the exchange begins.

DORA Process

7. Discover: the client broadcasts a DHCPDISCOVER message to locate available servers.
8. Offer: the server proposes an address and configuration values in a DHCPOFFER message.
9. Request: the client broadcasts a DHCPREQUEST identifying the selected offer.
10. Acknowledge: the server confirms the lease with DHCPACK.

Project Behavior

The configured pool begins at 192.168.10.100 and ends at 192.168.10.150. The lease screenshot later in this report shows PC2 receiving 192.168.10.100, with gateway and DNS both set to 192.168.10.1. That result proves that the DHCP options were delivered in addition to the IP address itself.

Operational Importance

- Reduces configuration errors and duplicate addresses.
- Allows central changes to gateway, DNS, and lease policy.
- Supports temporary clients without consuming permanent static addresses.
- Uses broadcasts during initial discovery, making the protocol easy to observe in a packet capture.

A 12-hour lease is appropriate for this environment because the topology is used for short laboratory sessions. In a production environment, the lease time would be selected based on client turnover, subnet size, and service availability requirements.

7. Protocol Analysis: DNS

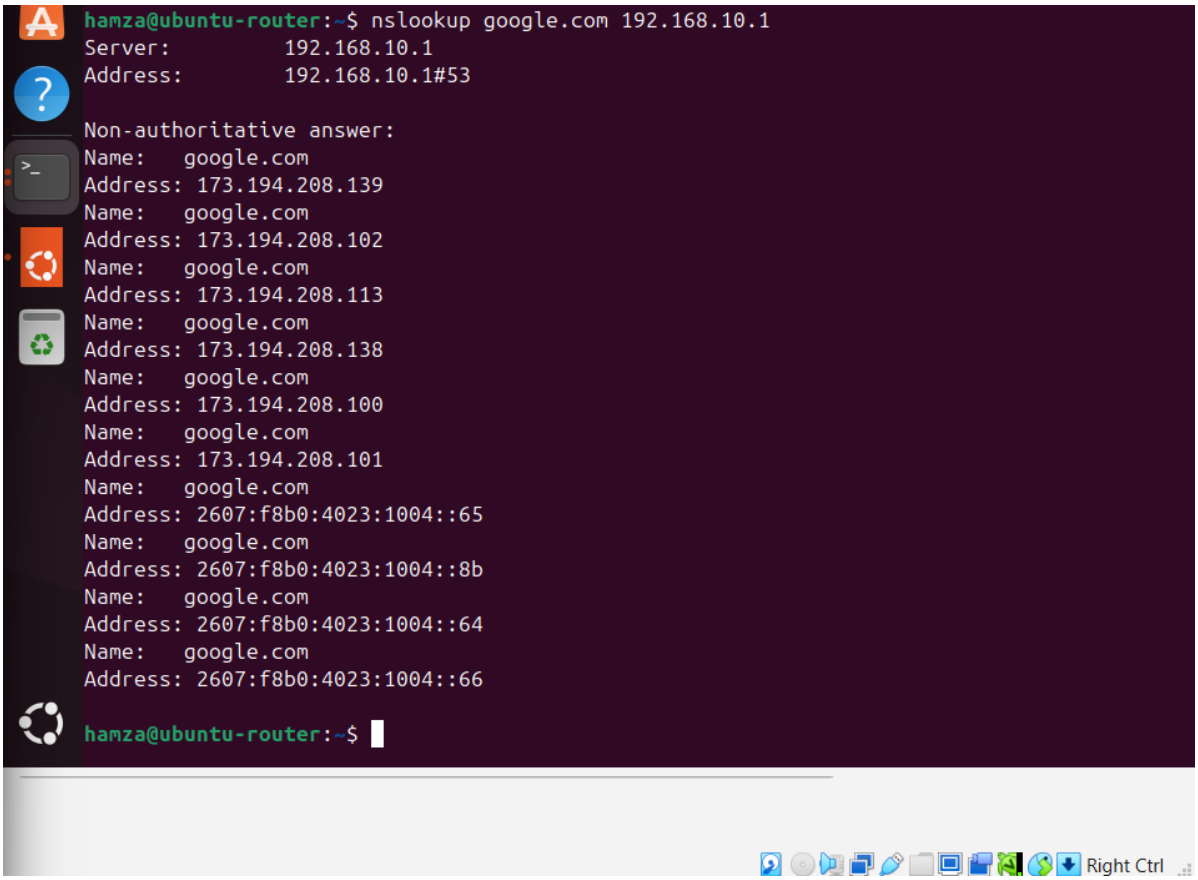

Figure 4. Successful nslookup of google.com through the local DNS forwarder at 192.168.10.1.

Domain Name System translates human-readable names into IP addresses. In this project, clients query 192.168.10.1, and dnsmasq forwards external questions to the explicitly configured upstream resolvers. The nslookup output identifies 192.168.10.1 as the responding server and returns multiple IPv4 and IPv6 records for google.com.

DNS typically uses UDP port 53 for ordinary queries because UDP minimizes overhead. TCP port 53 is used when responses are too large, when reliability is required, or for operations such as zone transfers. The successful lookup confirms three conditions: the client can reach the local resolver, dnsmasq is running and listening, and the router has external connectivity to an upstream DNS service.

The appearance of several A and AAAA answers demonstrates that one domain can map to multiple addresses. This supports redundancy, geographic distribution, and load balancing. The lookup also directly supports the later ping google.com test because the client must resolve the name before it can send ICMP echo requests to the destination address.

8. Protocol Analysis: HTTP and Apache

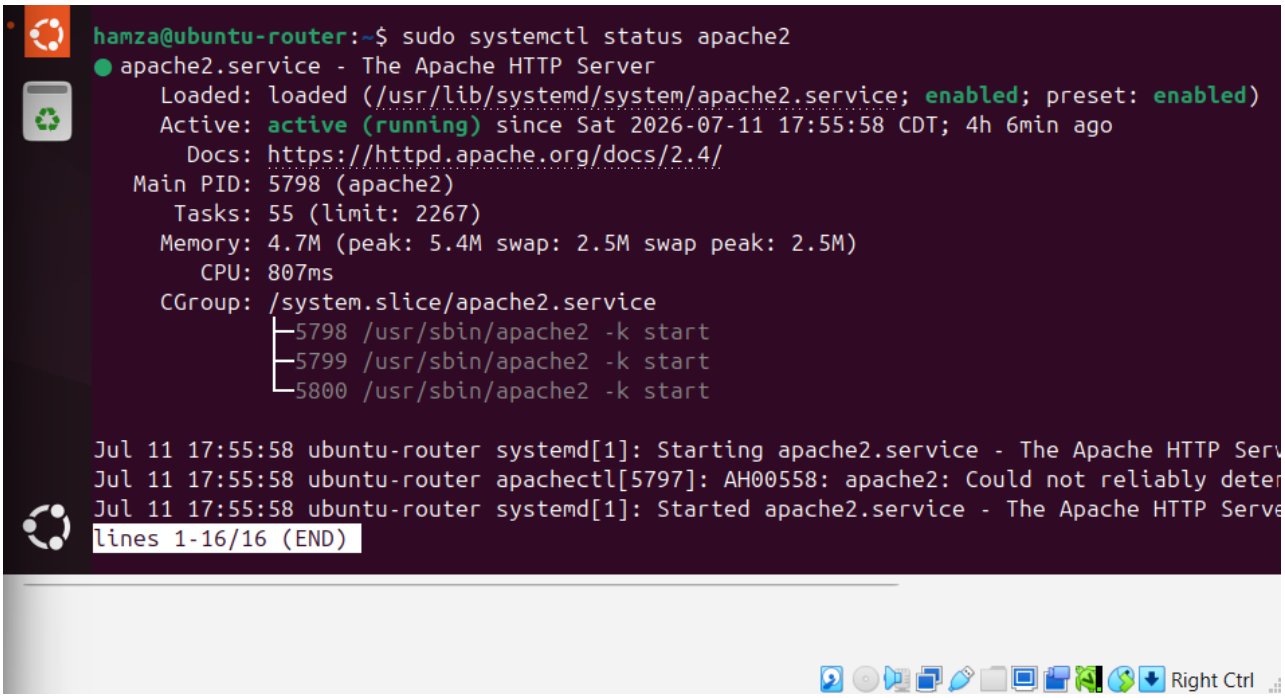

Figure 5. Apache2 service status showing the HTTP server enabled and active (running).

Hypertext Transfer Protocol provides application-layer communication between web clients and servers. Apache2 was installed on the Ubuntu VM and configured to start as a system service. The status output shows that apache2.service is loaded, enabled, and active. This confirms both the current runtime state and automatic startup behavior.

HTTP normally operates over TCP port 80. TCP is appropriate because a browser must receive web content in the correct order and must detect missing data. A browser begins with a TCP three-way handshake, sends an HTTP request such as GET, receives a status line and content, and then closes or reuses the TCP connection.

The default Apache page is sufficient as a validation target because it proves that the web daemon accepted a connection, processed the request, and returned an HTML response. In a production deployment, the default page would be replaced, access would normally use HTTPS, and firewall policy would be limited to the required ports.

9. HTTP Browser Validation

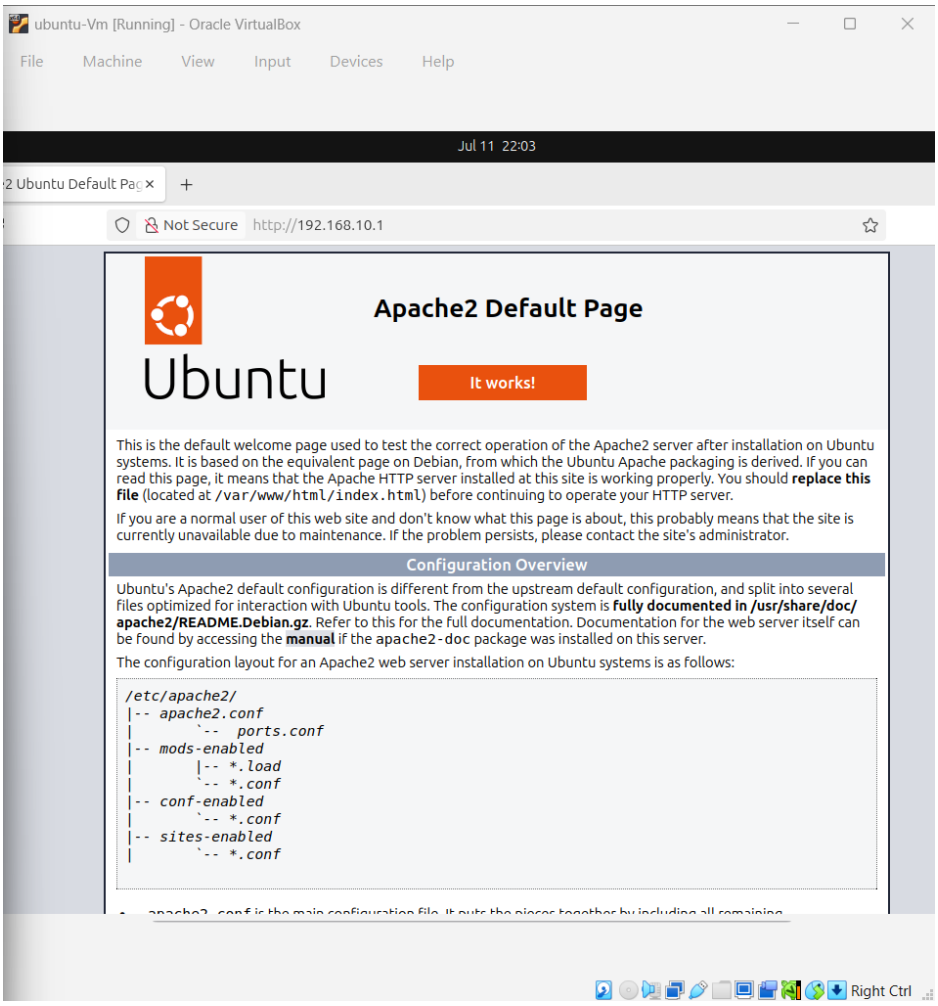

Figure 6. Apache2 default page loaded in Firefox from the Ubuntu web server.

The browser test provides end-to-end application validation. A successful page load requires more than an active Apache process: the client must have a valid IP configuration, the Layer 2 path through SW1 must work, ARP must resolve the local destination or gateway address, TCP must establish a connection, and HTTP must return the webpage content.

The visible Apache2 Default Page confirms that the web server responded successfully. This result verifies the application layer while indirectly confirming the lower layers that support it. It is therefore stronger evidence than a service-status command alone.

Observed Result

- Firefox reached the web service at the router/server address.
- The page rendered without a connection timeout or refusal.
- Apache returned the expected default HTML content.
- The test confirmed local LAN connectivity and TCP/HTTP operation.

A useful next step for a production-quality system would be to create a custom project homepage, enable TLS with HTTPS, and restrict administrative services from the client segment. Those enhancements were outside the core requirements of this laboratory but follow naturally from the completed design.

10. Protocol Analysis: ICMP and Connectivity Testing

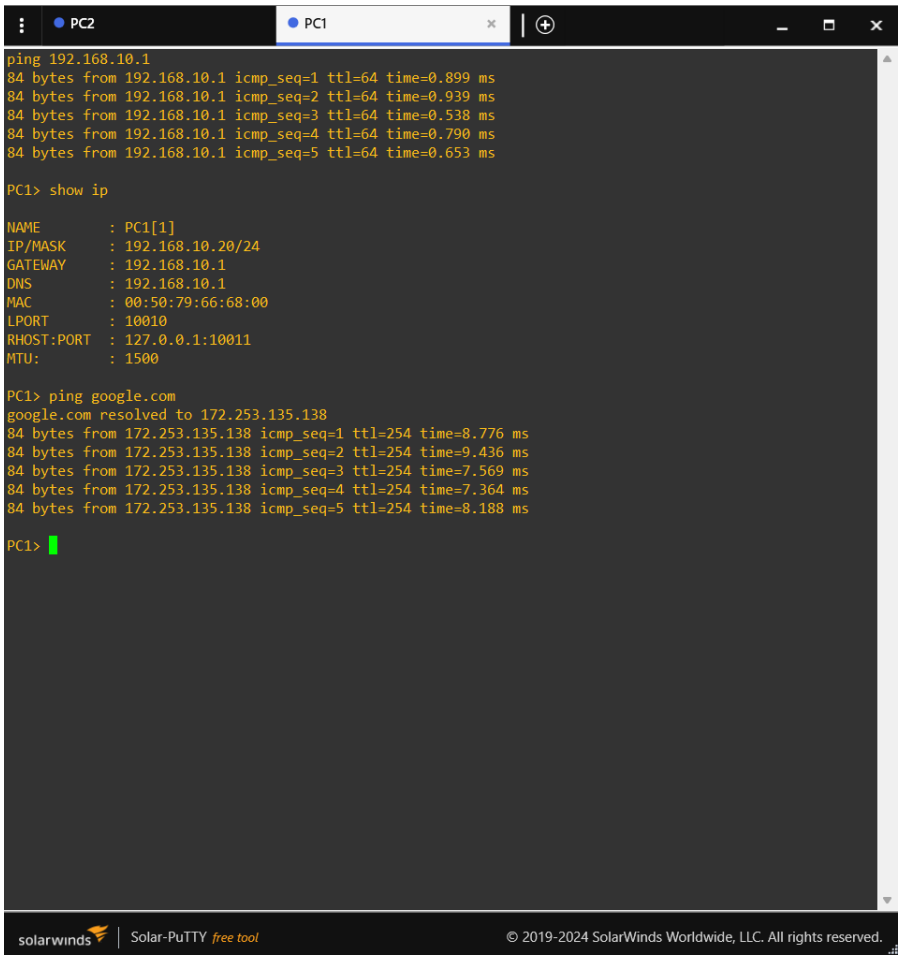

Figure 7. PC1 addressing, successful gateway ping, DNS resolution, and Internet ICMP replies.

Internet Control Message Protocol is used for network diagnostics and error reporting. The ping utility sends ICMP Echo Request messages and expects Echo Reply messages. PC1 first pings 192.168.10.1 and receives five replies with sub-millisecond response times, confirming local connectivity to the Ubuntu gateway.

The show ip output verifies PC1 as 192.168.10.20/24 with gateway and DNS both set to 192.168.10.1. The subsequent ping google.com resolves the name to 172.253.135.138 and receives five replies. This single test demonstrates DNS resolution, routing, NAT, external connectivity, and ICMP return traffic.

Test	Expected Result	Observed Result	Status
Ping 192.168.10.1	Replies from local gateway	5 replies, TTL 64, <1 ms	Pass
show ip	Correct address, gateway, and DNS	192.168.10.20/24; gateway/DNS .1	Pass
Ping google.com	Name resolves and external replies return	Resolved to 172.253.135.138; 5 replies	Pass

The different TTL values are expected. The local Ubuntu gateway replies with a typical Linux TTL of 64, while the external destination's packets arrive with a different remaining TTL after traversing multiple routers.

11. Protocol Analysis: ARP

Address Resolution Protocol maps an IPv4 address to a Layer 2 MAC address on the local network. Before PC1 can send an Ethernet frame to 192.168.10.1, it must know the gateway interface's MAC address. If the mapping is not already cached, PC1 broadcasts an ARP request asking which device owns 192.168.10.1. The router responds with an ARP reply containing its MAC address.

ARP is essential because IP addressing and Ethernet addressing serve different purposes. IP addresses identify logical network locations, while MAC addresses identify interfaces within the local broadcast domain. The switch forwards frames based on learned MAC addresses, not based on IP addresses.

ARP in This Project

- PC1 and PC2 use ARP to communicate with 192.168.10.1.
- Hosts on the same /24 subnet can ARP directly for each other.
- Traffic to an external network is framed to the default gateway's MAC address, not to the remote host's MAC address.
- The tcpdump screenshot includes ARP requests and replies, confirming live Layer 2 address resolution.

ARP traffic is not routed beyond the local subnet. This behavior keeps local hardware-address discovery within the broadcast domain. It also means that incorrect subnet masks can cause failures: a host may attempt to ARP for a remote address instead of forwarding the packet to the gateway.

From a security perspective, ARP has no built-in authentication and can be abused through spoofing. Production networks may use switch protections such as Dynamic ARP Inspection, DHCP snooping, static bindings for critical systems, and segmentation to reduce this risk.

12. Protocol Analysis: TCP and UDP

TCP and UDP are transport-layer protocols that support different application requirements. Both identify application endpoints using port numbers, but they provide different delivery behavior.

Characteristic	TCP	UDP
Connection model	Connection-oriented	Connectionless
Reliability	Acknowledgments, retransmission, sequencing	No built-in acknowledgment or retransmission
Ordering	Maintains byte-stream order	Datagrams may arrive out of order
Overhead	Higher	Lower
Project examples	HTTP web access; possible large DNS responses	DHCP and ordinary DNS queries
Typical use	Web, file transfer, remote login	Real-time traffic, discovery, lightweight queries

TCP Operation

A TCP session normally begins with SYN, SYN-ACK, and ACK. Sequence numbers and acknowledgments allow the receiver to reconstruct the byte stream and request retransmission when data is lost. These properties make TCP suitable for HTTP, where missing or out-of-order HTML content would be unacceptable.

UDP Operation

UDP sends independent datagrams without creating a session. DHCP relies on UDP because a new client cannot establish an ordinary IP-based connection before it has an address. DNS usually relies on UDP because most questions and answers are small and can be completed with one request and one response. Applications may add their own retry logic when needed.

The project therefore illustrates why network applications select transport services according to their needs rather than using one transport protocol for every task.

13. DHCP Lease Verification on PC2

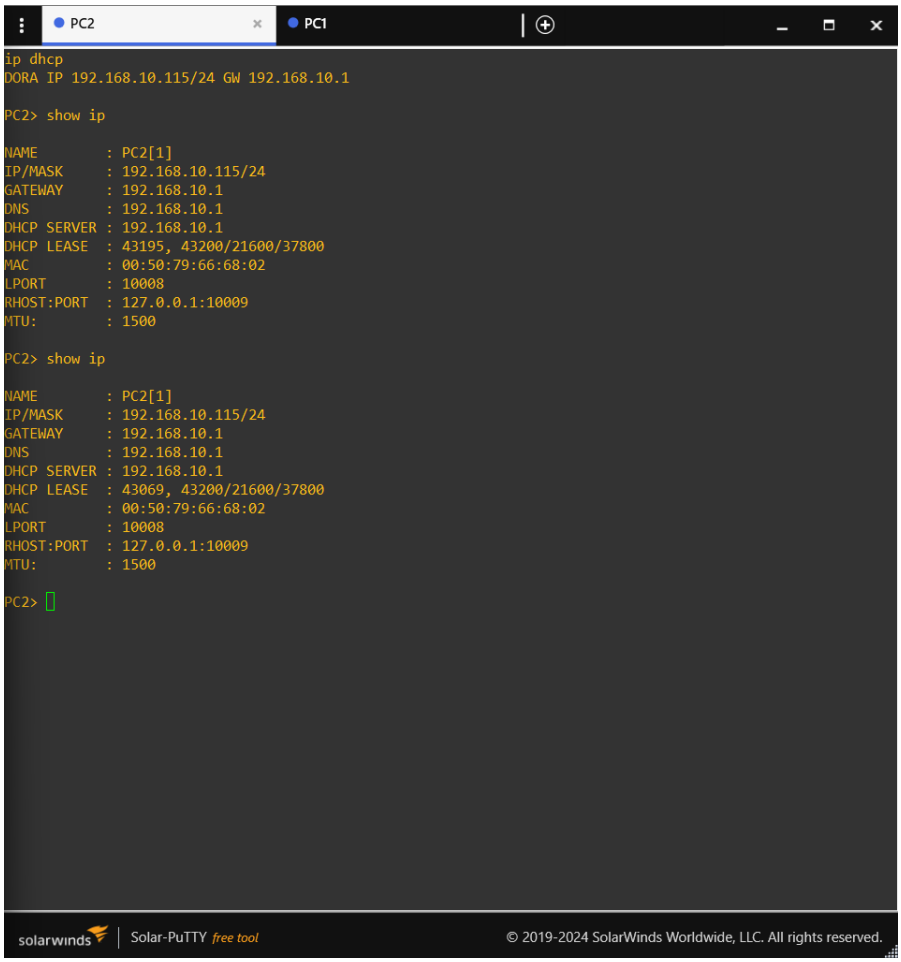

Figure 8. PC2 receiving a dynamic lease and displaying the assigned network parameters.

PC2 was used to verify dynamic address assignment. The DHCP command obtains a lease from the Ubuntu dnsmasq server, and show ip displays the resulting configuration. The visible address is 192.168.10.100/24, which is the first address in the configured 192.168.10.100–150 pool. The gateway and DNS values are both 192.168.10.1, matching DHCP options 3 and 6.

Parameter	Expected	Observed	Assessment
Client address	Within 192.168.10.100–150	192.168.10.100	Correct
Subnet prefix	/24	/24	Correct
Default gateway	192.168.10.1	192.168.10.1	Correct
DNS server	192.168.10.1	192.168.10.1	Correct
Lease delivery	DHCP success message	DHCP configuration displayed	Correct

This test is important because it verifies the complete DHCP transaction from the client's perspective. A server configuration file may look correct while the service is stopped, bound to the wrong interface, or blocked by a firewall. Receiving and displaying the lease proves that the service is reachable and that its options are usable by a client.

14. Router Interface Verification

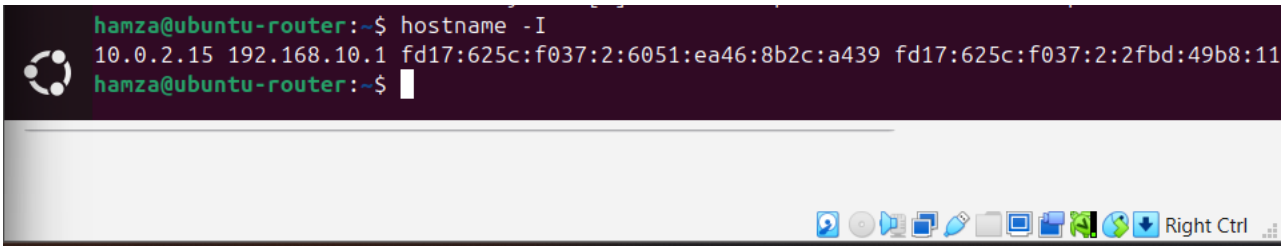

Figure 9. hostname -I output confirming the Ubuntu router's internal address of 192.168.10.1.

The hostname -I command lists the IP addresses assigned to the Ubuntu VM. The output includes 192.168.10.1, confirming that the internal interface has the expected gateway address. Additional addresses shown belong to other interfaces or address families used by the virtual machine and its upstream connectivity.

The internal address is foundational to the entire project. DHCP advertises it as the gateway and DNS server, clients send local DNS queries to it, NAT forwarding begins there, Apache listens on the host, and ARP requests identify the corresponding interface MAC address. A mismatch at this step would cause several later tests to fail simultaneously.

Verification Checklist

- The address belongs to the planned 192.168.10.0/24 network.
- The client gateway and DNS settings point to the same address.
- The interface is reachable from PC1 with ICMP.
- dnsmasq is configured to listen on this address.
- The address serves as the local HTTP destination.

15. Packet Capture and Results Analysis

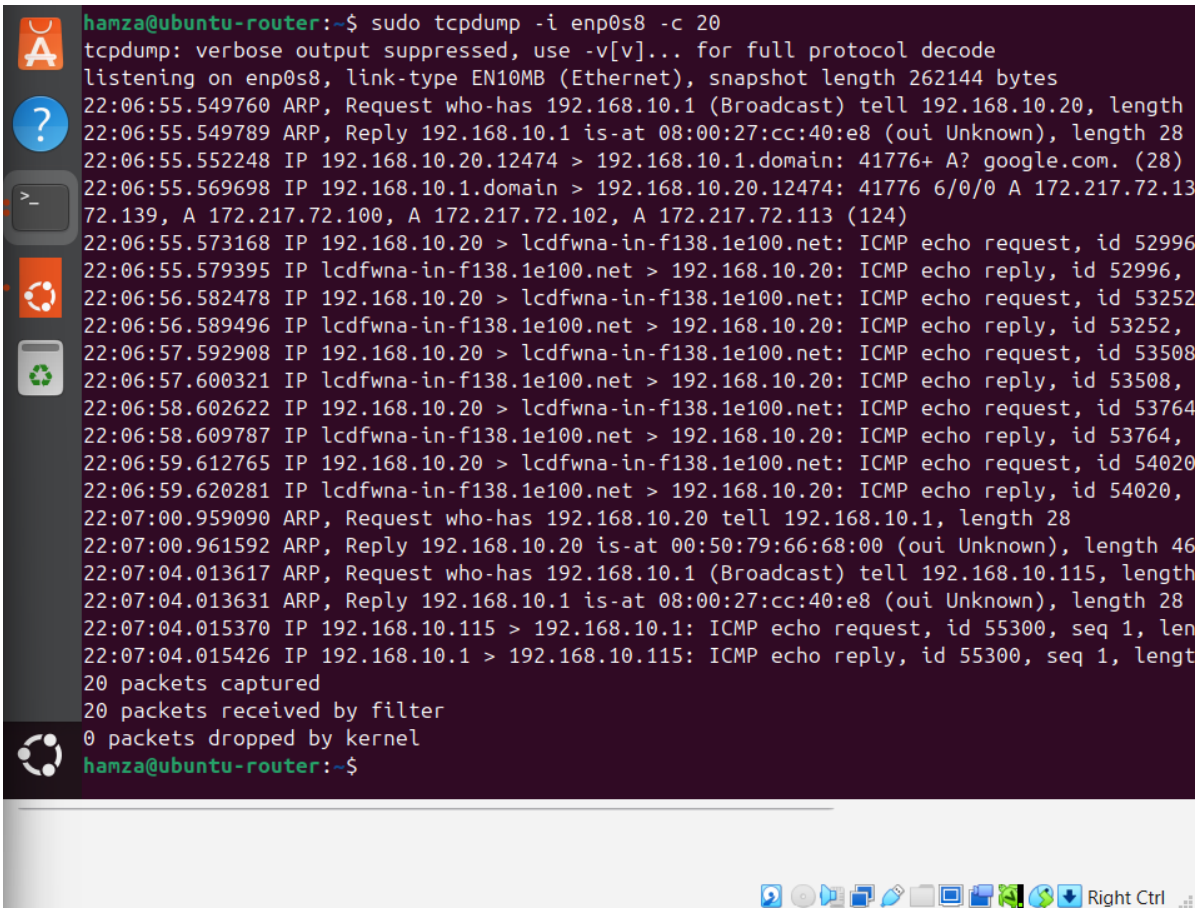

Figure 10. tcpdump capture on enp0s8 displaying ARP exchanges and ICMP echo traffic.

tcpdump was executed on the internal interface with numeric output and a capture limit. The screenshot shows ARP requests and replies involving 192.168.10.1 and client addresses, followed by ICMP echo requests and replies. This is direct packet-level evidence that the protocols discussed in the report were active on the LAN.

Evidence in Capture	Interpretation
ARP request for 192.168.10.1	A client needs the gateway's MAC address before sending an Ethernet frame.
ARP reply from the router	The gateway identifies its MAC address to the requester.
ICMP echo request	A host is testing reachability with ping.
ICMP echo reply	The destination received the request and returned a response.
Packets captured by filter	The capture was taken on the correct internal interface.

Packet capture is valuable because it reveals what occurs below the application output. A successful ping result shows that communication worked; tcpdump shows the supporting ARP resolution and the individual ICMP messages. In troubleshooting, this distinction helps determine whether a problem exists at the application, transport, network, or data-link layer.

16. Integrated Testing and Results

Requirement	Test Method	Evidence	Result
Topology operational	Inspect GNS3 links and node states	Figure 1	Pass
NAT configured	Inspect iptables forwarding and POSTROUTING rules	Figure 2	Pass
DHCP/DNS configured	Inspect dnsmasq configuration	Figure 3	Pass
DNS resolution	Run nslookup google.com	Figure 4	Pass
Apache running	Run systemctl status apache2	Figure 5	Pass
HTTP reachable	Open Apache page in Firefox	Figure 6	Pass
PC1 configured	Run show ip	Figure 7	Pass
Internet reachable	Ping google.com	Figure 7	Pass
DHCP lease delivered	Run dhcp and show ip on PC2	Figure 8	Pass
Router address correct	Run hostname -I	Figure 9	Pass
ARP/ICMP observable	Run tcpdump on enp0s8	Figure 10	Pass

The test sequence moves from lower-level prerequisites toward application services. Addressing and gateway reachability were verified first, followed by DNS, Internet access, Apache status, browser access, and packet inspection. This layered approach reduces troubleshooting time because a failure can be isolated before moving to the next dependency.

All documented requirements produced successful results. The evidence is internally consistent: the same 192.168.10.1 address appears as the router interface, client gateway, client DNS server, dnsmasq listening address, and local service host.

17. Challenges, Lessons Learned, and Conclusion

Key Challenges

- Selecting the correct Ubuntu interfaces for the internal LAN and the upstream connection.
- Ensuring that Linux IP forwarding and iptables rules worked together rather than configuring NAT alone.
- Binding dnsmasq to the internal interface so it served the GNS3 clients without interfering with other host networking.
- Keeping static addresses separate from the DHCP pool to avoid conflicts.
- Testing services in a logical order so that DNS or HTTP symptoms were not mistaken for basic routing failures.

Lessons Learned

The project showed that network services are highly interdependent. A browser page may depend on ARP, IP addressing, routing, TCP, and HTTP. An Internet ping by name additionally depends on DNS and NAT. Configuration commands are therefore only one part of network engineering; systematic verification and packet analysis are equally important.

Conclusion

The final GNS3 environment successfully implemented a small virtual LAN with centralized routing and services. The Ubuntu VM operated as the 192.168.10.1 gateway, DHCP and DNS server, NAT router, and Apache web server. PC1 demonstrated correct static addressing and Internet reachability, while PC2 demonstrated successful DHCP assignment. nslookup verified DNS forwarding, Firefox verified HTTP service, and tcpdump verified ARP and ICMP exchanges. Together, the results satisfy the project objectives and provide a complete practical demonstration of the principal protocols covered in CSCE 3530.

References

IETF. RFC 2131, Dynamic Host Configuration Protocol, March 1997.

IETF. RFC 1034 and RFC 1035, Domain Names—Concepts, Facilities, Implementation, and Specification, November 1987.

IETF. RFC 3022, Traditional IP Network Address Translator (Traditional NAT), January 2001.

IETF. RFC 9110, HTTP Semantics, June 2022.

IETF. RFC 792, Internet Control Message Protocol, September 1981.

IETF. RFC 826, An Ethernet Address Resolution Protocol, November 1982.

IETF. RFC 9293, Transmission Control Protocol (TCP), August 2022.

IETF. RFC 768, User Datagram Protocol, August 1980.

Kurose, J. F., and Ross, K. W. Computer Networking: A Top-Down Approach. Pearson.

GNS3 Documentation; Ubuntu Server Documentation; dnsmasq, iptables, Apache2, and tcpdump manual pages.